INSY7314 POE Part 1 submission

HUSTLEHUB+ PART 1

ST10445478 - Christopher Doyle
ST10446382 – Pabi Kerns
ST10446445 - Daniel Wilson

Table of Contents

1. GitHub link: *2*

2. Architecture Diagram *3*

3. API and Postman Screenshots *4*

Screenshot 1 - Successful Registration **4**

Screenshot 2 – Successful Login and JWT **5**

Screenshot 3 - Protected Profile **6**

Screenshot 4 - Unauthorised Request **7**

Screenshot 5 - Duplicate Email **8**

Screenshot 6 - Invalid Input **9**

Screenshot 7 - Postman Collection Run **10**

Screenshot 8 – HTTPS **11**

4. Demonstration Video *12*

1. GitHub link:

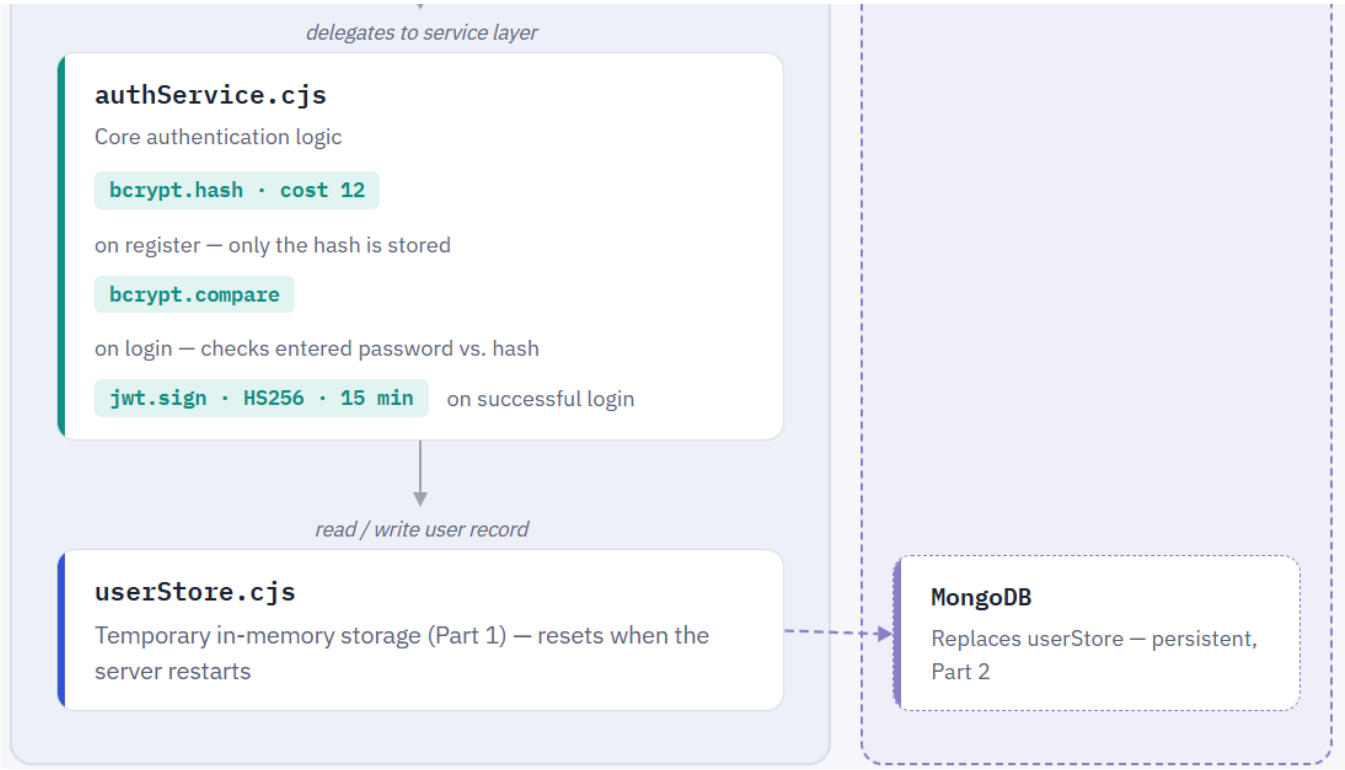
https://github.com/EMWCCN/insy7314-2026-g3-insy7314-2026-hustlehub-plus-part1-christopher-doyle.git

2. Architecture Diagram

Insert the final HustleHub+ architecture diagram below. The diagram should clearly show the main system parts and where the security controls fit into the request flow.

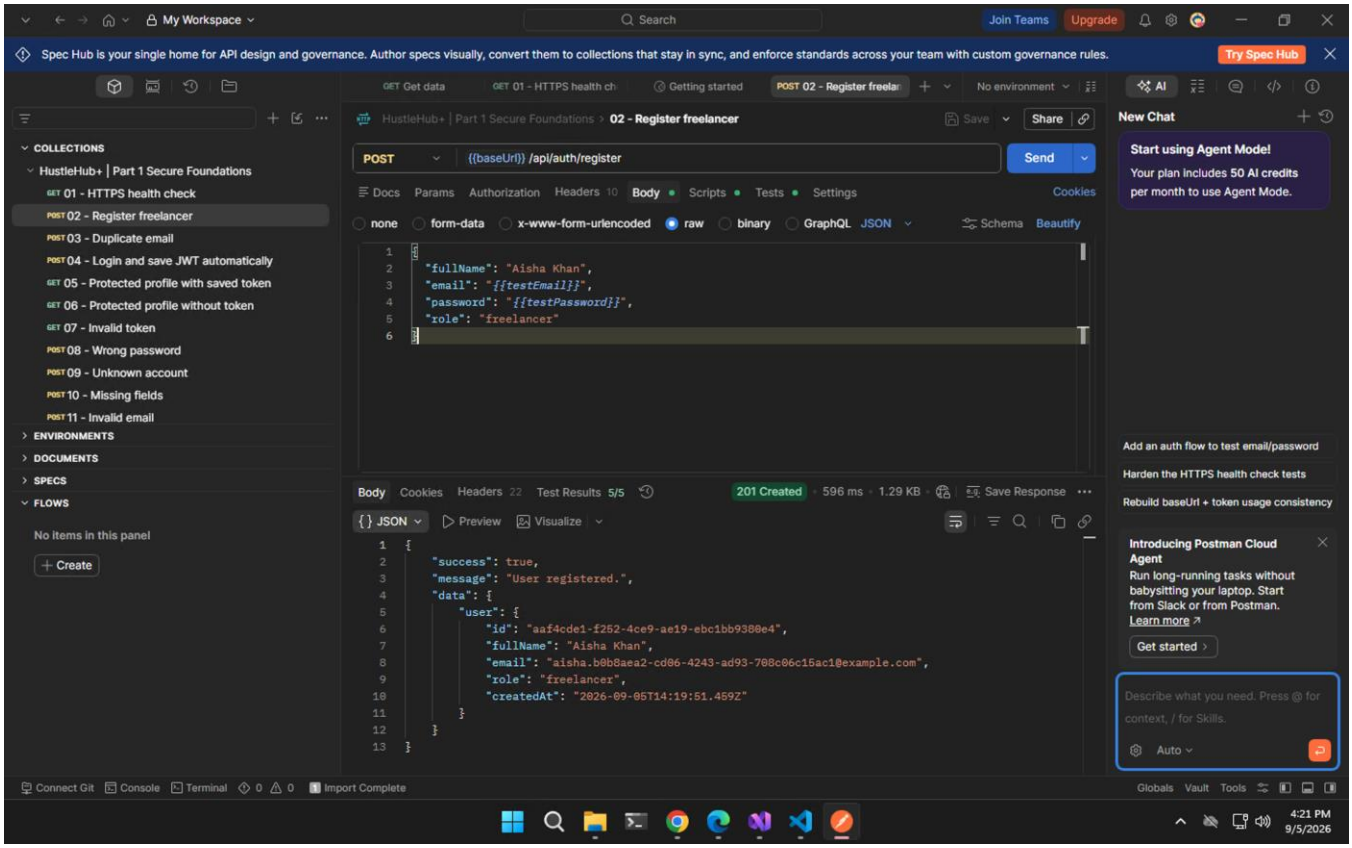
- Client / Postman sending requests over HTTPS
- Node.js and Express backend
- Routes, controllers, services and middleware
- Input validation before authentication logic
- bcrypt password hashing for registration and login
- JWT generation and JWT verification for protected routes
- Temporary in-memory user storage for Part 1
- MongoDB and React marked as later-stage components if they are shown


PART 1 — LIVE IN THIS DEMO
Client / Postman
Sends registration, login & profile requests
extends client
PART 2 — LATER STAGE
React frontend
Replaces Postman as the client — Part 2
HTTPS — TLS-encrypted request
server.cjs → app.cjs
Node.js + Express — HTTPS listener on :4000 (local SSL cert)
Global middleware: Helmet · JSON parser · rate limit (30 req / 15 min)
routed to matching endpoint
authRoutes.cjs
POST /register
POST /login
GET /me — protected
PUBLIC — REGISTER & LOGIN
PROTECTED — GET /ME
validateAuthInput.cjs
Validates input before auth logic runs
types · allowed fields · rejects operator injection
authMiddleware.cjs
Verifies JWT before controller runs
signature · expiry · algorithm · issuer · audience
authController.cjs
Handles request / response — delegates logic to the service layer
delegates to service layer
authService.cjs
Core authentication logic
bcrypt.hash · cost 12
on register — only the hash is stored
bcrypt.compare
on login — checks entered password vs. hash
jwt.sign · HS256 · 15 min
on successful login


delegates to service layer
authService.cjs
Core authentication logic
bcrypt.hash · cost 12
on register — only the hash is stored
bcrypt.compare
on login — checks entered password vs. hash
jwt.sign · HS256 · 15 min
on successful login
read / write user record
userStore.cjs
Temporary in-memory storage (Part 1) — resets when the server restarts
MongoDB
Replaces userStore — persistent, Part 2

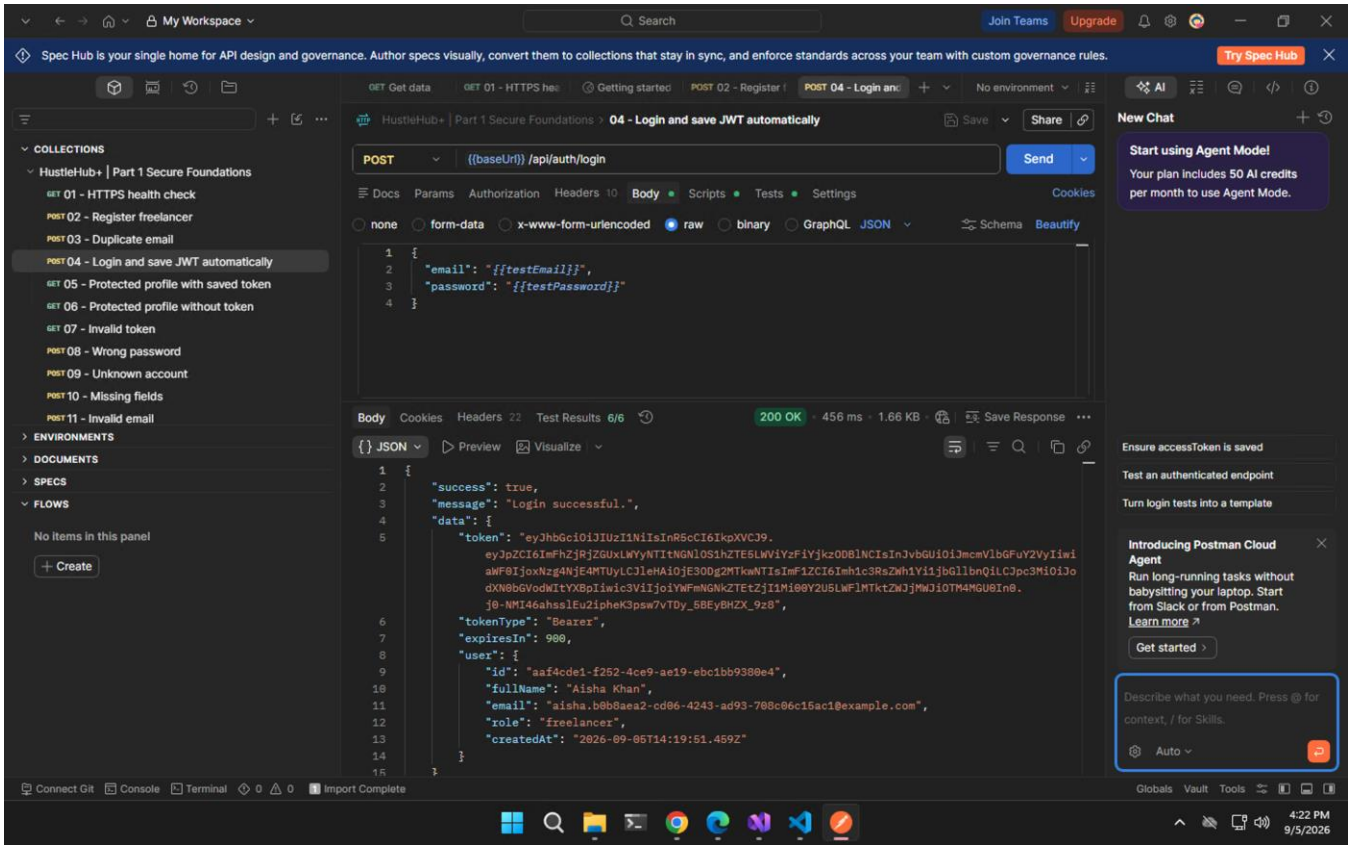
3. API and Postman Screenshots

Screenshot 1 - Successful Registration



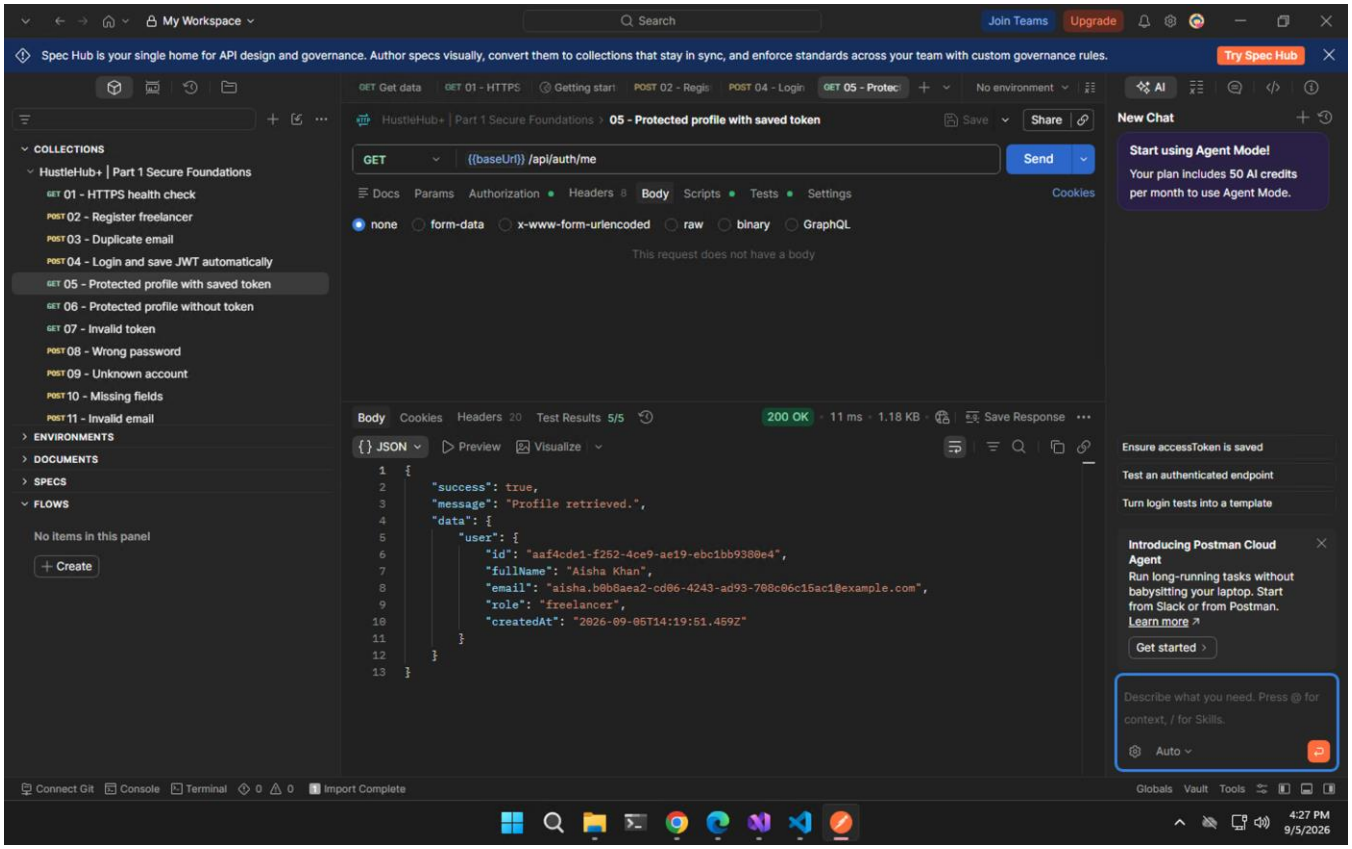
Screenshot 1 shows a successful registration request sent to the POST /api/auth/register endpoint. The client sends a JSON payload containing the user's name, email, password and role (freelancer). Once the backend validates the inputs, it securely hashes the password using bcrypt before saving the account to memory. The API then returns a 201 Created status with the response showing basic account details while keeping sensitive credentials hidden.

Screenshot 2 – Successful Login and JWT



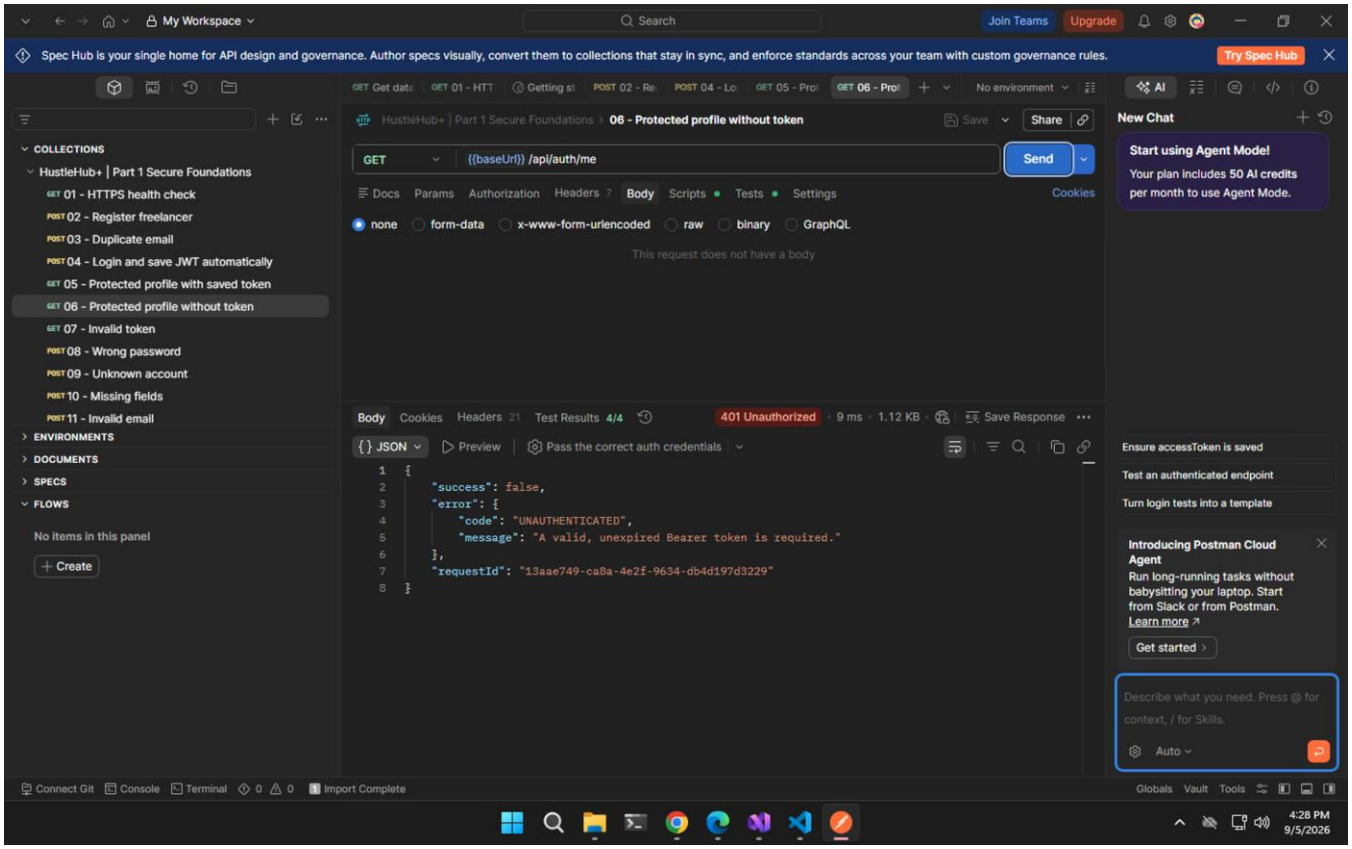
Screenshot 2 shows a successful login request sent to the POST /api/auth/login endpoint. The client submits the user's email and password in the JSON body. After validating the credentials against the saved bcrypt hash, the server generates a signed JSON Web Token (JWT) and returns a 200 OK response. The Postman test script automatically captures and saves this Bearer token for subsequent authenticated requests.

Screenshot 3 - Protected Profile



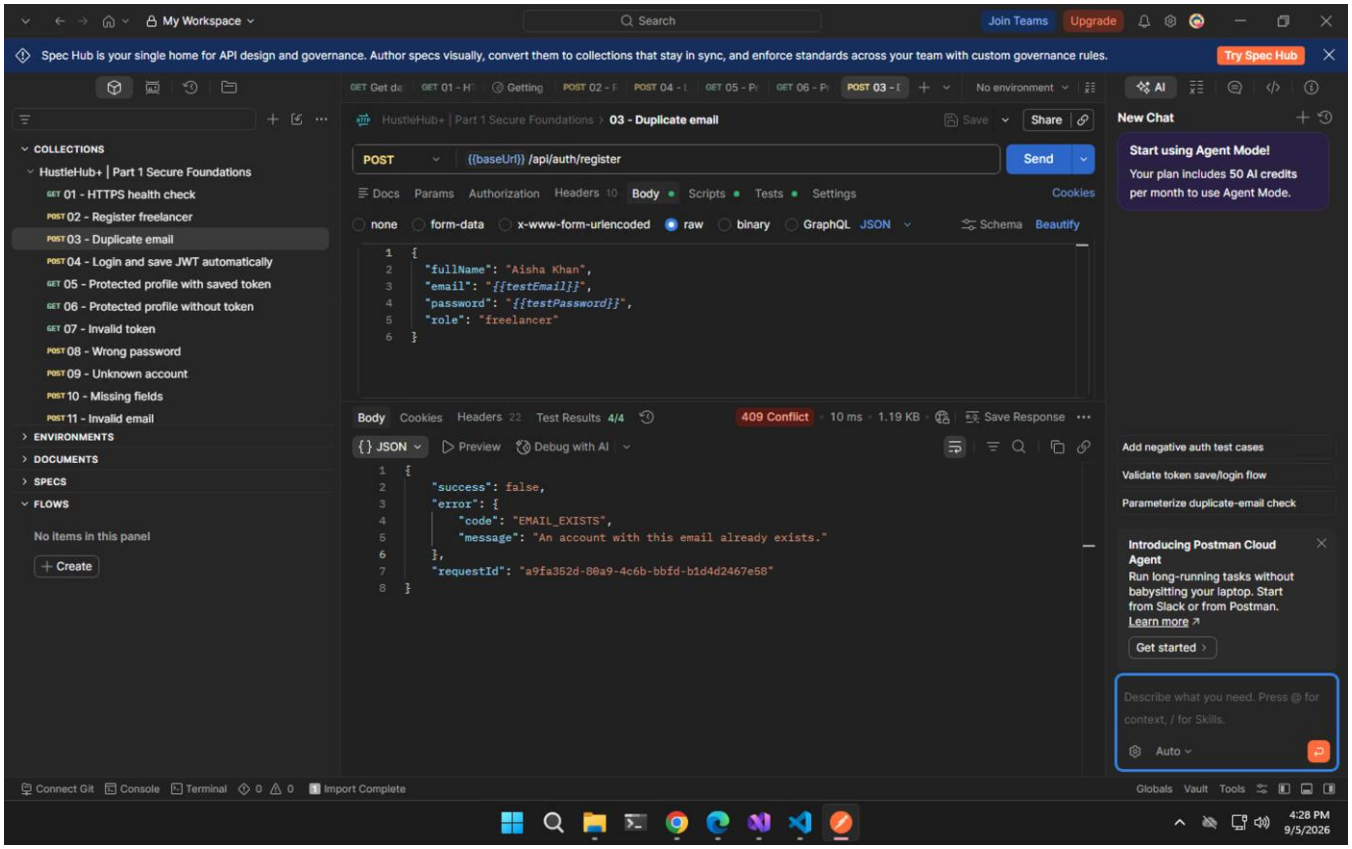
Screenshot 3 shows an authenticated request sent to the protected GET /api/auth/me route. The request includes the saved JWT inside the Authorization: Bearer <token> header. The authentication middleware verifies the token's signature and expiration before allowing access. Because the token is valid, the server returns a 200 OK status along with the logged-in user's profile information.

Screenshot 4 - Unauthorized Request



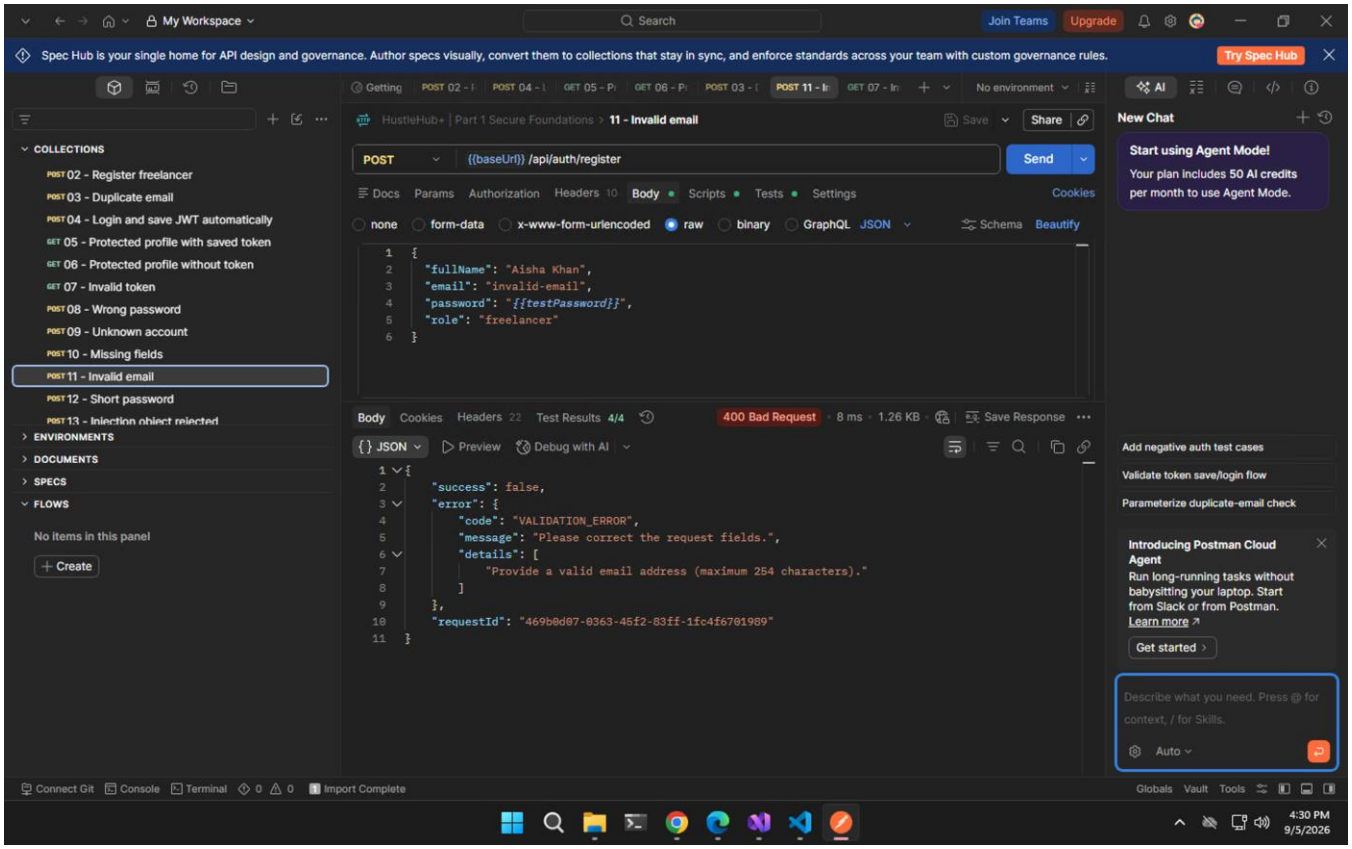
Screenshot 4 shows an unauthenticated request sent to the same GET /api/auth/me route, but without providing a Bearer token. The authentication middleware intercepts the request and blocks access to the user profile before it reaches the controller. The API returns a 401 Unauthorized status code with an error message stating that a valid, unexpired Bearer token is required.

Screenshot 5 - Duplicate Email



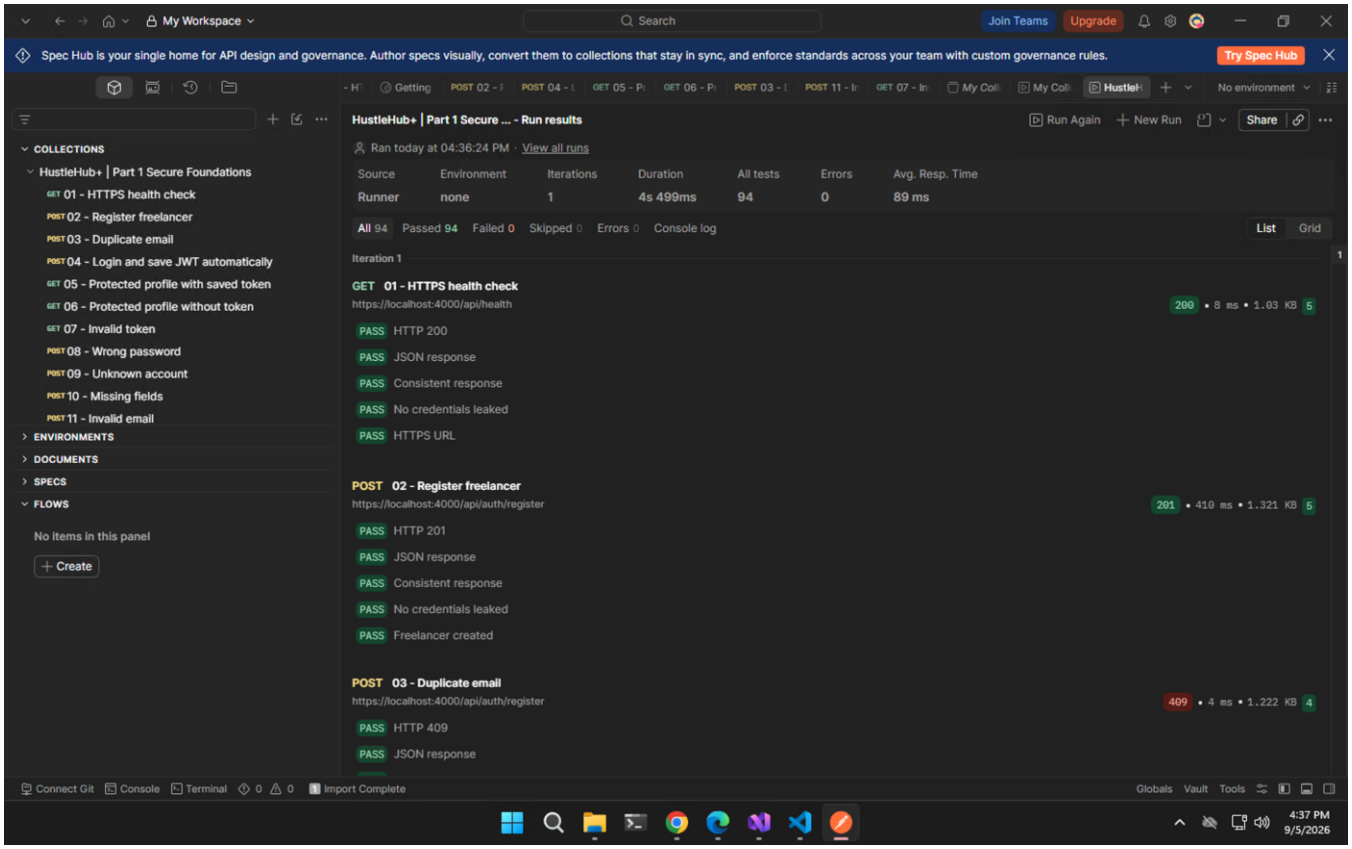
Screenshot 5 shows a registration attempt sent to POST /api/auth/register using an email address that already exists in the system. The backend checks existing user records before creating new accounts. When it detects a duplicate email, it rejects the registration and returns a 409 Conflict status code along with an EMAIL_EXISTS error message.

Screenshot 6 - Invalid Input



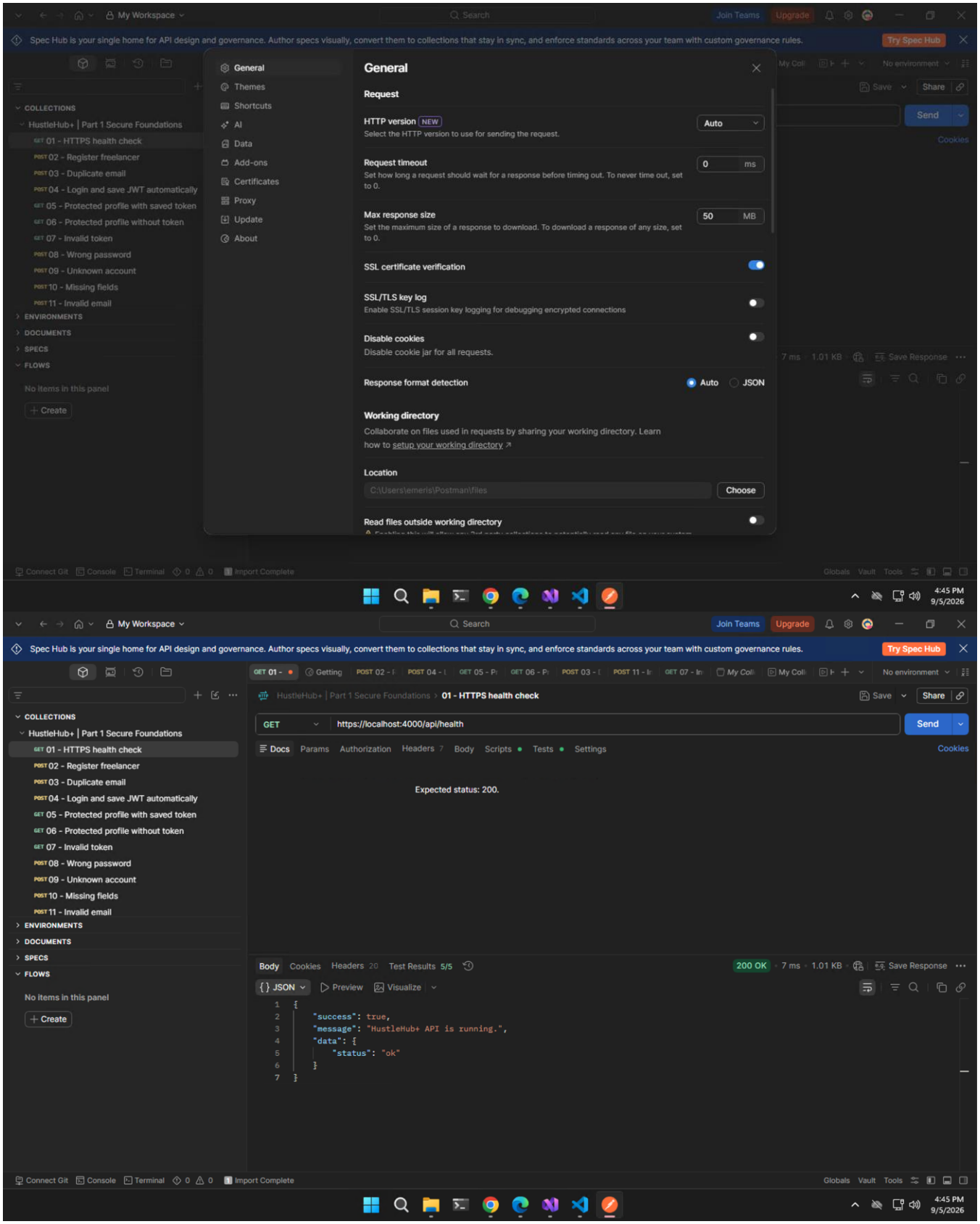
Screenshot 6 demonstrates input validation when sending malformed data to POST /api/auth/register. In this test, an invalid string (invalid-email) is submitted in the email field. The validation middleware catches the malformed value before processing the request, returning a 400 Bad Request status with a VALIDATION_ERROR payload that explicitly explains what needs to be fixed.

Screenshot 7 - Postman Collection Run



Screenshot 7 shows the full test suite execution using the Postman Collection Runner. A total of 94 automated assertions were run across all backend endpoints, covering health checks, user registration, authentication flows, protected profile access and various negative error scenarios. All 94 tests passed successfully with zero failures, confirming that the API, validation middleware and security controls are working as intended.

Screenshot 8 – HTTPS



Screenshot 8 demonstrates secure HTTPS communication with the local backend server. The top image confirms that Postman's strict SSL certificate verification setting is enabled. The bottom screenshot shows a successful health check request sent over HTTPS to `https://localhost:4000/api/health` returning a `200 OK` status, proving that the Node.js server correctly serves encrypted requests using self-signed TLS certificates.

4. Demonstration Video

Upload the final demo video to YouTube and paste the link below.

YouTube link: https://youtu.be/bI21BFtY164